Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 00 (2026) 000–000

Procedia Computer Science

www.elsevier.com/locate/procedia

38th European Modeling & Simulation Symposium (EMSS 2026), I3M 2026

A Controlled-Replay Prototype for Auditable Versioned Financial Event Graph Updates

Jiakai Tong[a], Yue Zhao[a]

[a]*School of Astronautics, Harbin Institute of Technology, Harbin, China*

Abstract

Financial event records used in simulation and audit workflows require evidence links, update histories, and reproducible state transitions. This paper presents a controlled-replay prototype for auditable versioned financial event graph updates. The artifact checks required event-record fields, enforces exact evidence-span containment, applies deterministic graph update operators, and records append-only version logs. Evaluation is intentionally narrow: 70 controlled records exercise oracle-controlled replay, a metadata-hidden diagnostic obtains 0.586 operator agreement, 20 FewFC-derived public mini-case records provide an external sanity check, and a 560-record scale sanity run checks lightweight runtime behavior. The paper reports no extraction F1, no market prediction, and no benchmark-level generalization. The evidence supports a limited auditability claim: controlled financial event updates can be replayed into inspectable graph transitions with explicit evidence and failure logs.

© 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the 23rd International Multidisciplinary Modeling & Simulation Multiconference.

Keywords: event graph; controlled replay; discrete-event simulation; auditability; version log

1877-0509 © 2026 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the scientific committee of the 23rd International Multidisciplinary Modeling & Simulation Multiconference.

1. Introduction

Financial event records are often treated as final structured outputs. For modeling and simulation workflows, that view is incomplete: a record may duplicate a prior disclosure, revise a previous amount, or contradict an earlier statement. If such changes are silently overwritten, later audit and verification become difficult. A simulation artifact should therefore preserve evidence, replay order, state transitions, and failure reasons.

Event records need replay and version logs because their operational meaning depends on graph state. The same incoming record may be a new event in an empty graph, a merge candidate after a target exists, or a failed operation if target resolution cannot be justified. A compact replay model can make these decisions inspectable by connecting each accepted record to a graph transition and each rejected record to an explicit skip reason.

Existing financial event extraction research focuses on converting documents into structured events [1, 2, 3]. Provenance and event-sourcing work emphasizes traceable state reconstruction [4, 5]. Discrete-event simulation and verification literature emphasizes explicit event-driven state change and validation discipline [6, 7, 8]. The gap addressed here is narrower: a reproducible artifact for evidence-gated replay of controlled financial event records into a versioned event graph.

This paper makes three contributions. First, it defines an evidence-gated controlled replay model for financial event records. Second, it implements versioned event graph transition artifacts, including target resolution, invariant checks, version logs, and replay traces. Third, it reports reproducible diagnostics, including metadata-hidden replay, negative controls, invariant checks, a public mini-case, and a scale sanity run.

2. Related work

2.1. Financial event extraction and event records

Document-level financial event extraction systems such as DCFEE [1] and Doc2EDAG [2] show that financial events often require multi-sentence argument assembly. FewFC provides a public few-shot financial event extraction dataset [3]. These works motivate structured financial event records. The present artifact starts after record construction and studies replay, evidence gates, and graph update logs rather than extraction model quality.

2.2. Provenance, audit trails, and event sourcing

The W3C PROV-O vocabulary defines provenance concepts for entities and activities [4]. Event sourcing records state changes as an append-only event history [5]. Database and workflow audit trails similarly emphasize traceable data evolution [9]. The prototype follows this auditability principle in a compact graph setting: each applied or failed replay decision is represented as a version log entry.

2.3. Discrete-event simulation and reproducible artifacts

Discrete-event simulation describes systems whose state changes at event times [6, 7]. Verification, validation, and testing guidance for simulation models stresses traceable assumptions, reproducible runs, and explicit limits on interpretation [8]. This paper uses those ideas for event graph maintenance: incoming records trigger state transitions, while replay traces support inspection and reproduction.

3. Methodology

3.1. Event records and evidence gate

An event record is represented as

$$x_i = (id_i, type_i, subj_i, obj_i, time_i, trig_i, span_i, doc_i, text_i, a_i, status_i). \tag{1}$$

The required fields are event identity, event type, subject, object, date, trigger, evidence span, source document identifier, and source text. Optional slots include amount and status. A record passes the evidence gate only when

$$span_i \neq \emptyset \quad \wedge \quad span_i \subset text_i, \tag{2}$$

where containment is exact string containment. The main experiments do not use fuzzy matching.

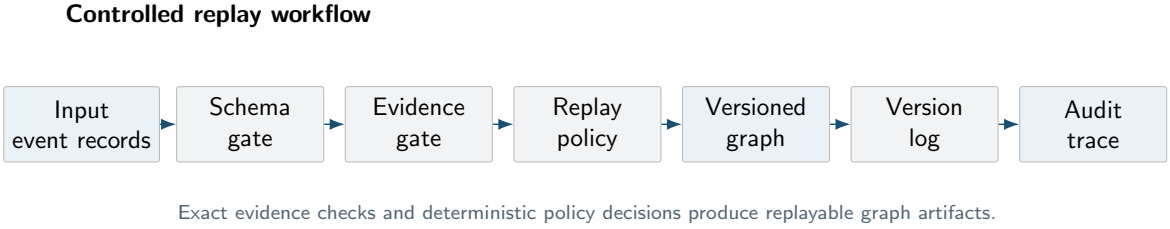

Fig. 1. Workflow for evidence-gated replay from event records to graph artifacts and audit traces.

3.2. Replay policy and target resolution

At replay step t, the simulator chooses an operator

$$o_t \in \{\texttt{ADD_EVENT}, \texttt{MERGE_EVENT}, \texttt{UPDATE_SLOT}, \texttt{MARK_CONFLICT}\}. \tag{3}$$

The oracle-controlled policy may read controlled metadata: perturbation type, expected operator, and base event identifier. The rule-based policy may read only event fields and current graph state. It cannot use perturbation labels, expected operators, gold grouping fields, or a direct base identifier as a target shortcut.

Target resolution is also split. Oracle target selection may use a base identifier only if that target exists in the graph. Rule-based target selection uses subject, event type, date proximity, amount, status, and textual update or conflict cues. If no target exists for a target-dependent operator, replay returns no target and records a failed operation.

3.3. Invariant checks and graph transition

The graph state is

$$G_t = (V_t, E_t, L_t), \tag{4}$$

where V_t contains event and entity nodes, E_t contains event-entity links, and L_t is an append-only version log. The transition is

$$G_t \xrightarrow{o_t} G_{t+1}. \tag{5}$$

The simulator also records a compact state summary

$$S_t = (g_t, n_t^e, n_t^v, n_t^m, n_t^u, n_t^c, n_t^{uc}), \tag{6}$$

where the terms denote graph version, active events, active entities, successful merges, successful slot updates, conflict marks, and unresolved conflicts. Failed target resolution is logged with `operation_applied=false`; it does not increment successful merge, update, or unresolved conflict counts.

3.4. Version log schema

Each version log entry stores the replay step, graph version, operator, incoming event identifier, target identifier when available, target-existence flag, applied flag, skip reason, source document identifier, and operation details. Unsupported metadata is logged with `skipped_reason=unsupported_or_unknown_metadata`. Figure 1 summarizes the replay workflow, and Figure 2 shows the state-transition view used by the simulator.

4. Experiments

4.1. Controlled oracle replay

The controlled stream contains 70 records derived from 30 deterministic seed records. It injects duplicate, update, conflict, and out-of-order replay perturbations. Table 1 reports schema validity, evidence coverage, replay completeness, and agreement with injected operator labels. These agreement values are controlled diagnostics, not extraction accuracy.

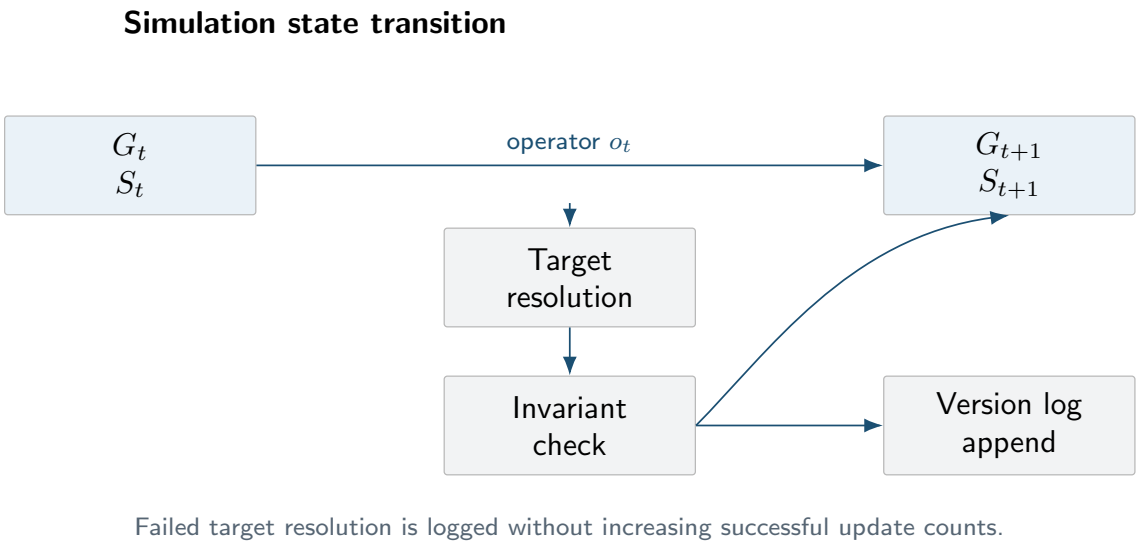

Fig. 2. State-transition view linking target resolution, invariant checks, graph update, and version-log append.

Table 1. Controlled replay diagnostics; agreement compares replay decisions with injected operator labels, not extraction accuracy.

Method	Schema	Evidence	Merge	Conflict	Update	Operator	Replay	n
Direct records	1.000	1.000	–	–	–	–	–	70
Schema gate	1.000	1.000	–	–	–	–	–	70
Evidence gate	1.000	1.000	–	–	–	–	1.000	70
Oracle-controlled	1.000	1.000	1.000	1.000	1.000	1.000	0.871	70

Table 2. Metadata-hidden diagnostic agreement; gold labels are used only after replay for comparison.

Setting	Metadata	Merge	Conflict	Update	Operator	Replay
Oracle-controlled	yes	1.000	1.000	1.000	1.000	0.871
Metadata-hidden	no	0.200	0.800	0.700	0.586	1.000

Table 3. FewFC-derived public mini-case, reported as an external sanity check only.

Records	Schema	Evidence	Replay	Active events	Logs
20	1.000	1.000	1.000	20	20

4.2. *Metadata-hidden rule-based replay*

The metadata-hidden diagnostic removes perturbation labels, expected operators, direct base identifiers, and gold grouping fields before policy decisions. Gold labels are restored only after replay to compute agreement. Table 2 shows that the metadata-hidden operator agreement is 0.586, exposing the current rule limits.

4.3. *Public mini-case*

The public mini-case uses 20 FewFC-derived records as an external sanity check only. It tests whether converted public financial event text can pass the same schema, exact evidence, and replay path. Table 3 reports this check. It is not a benchmark and does not report extraction F1.

4.4. *Negative controls and invariant checks*

Negative controls include missing required fields, evidence mismatches, invalid dates, and unsupported metadata. Table 4 reports rejection and logging behavior. Table 5 reports a separate small invariant check suite. These invariant

Table 4. Negative controls verify rejection and logging paths without successful graph updates.

Cases	Schema rej.	Evidence rej.	Bad dates	Unsupported logged	Crashes
20	10	5	5	5	0

Table 5. Invariant checks for failed target resolution and unsupported metadata.

Case	Expected behavior	Applied	Logged	Result
Missing target merge_event	log missing target without success count	false	true	pass
Missing target update_slot	log missing target without success count	false	true	pass
Missing target mark_conflict	log missing target without success count	false	true	pass
Invalid nested event	return schema error	false	true	pass
Unsupported metadata	log unsupported metadata without update	false	true	pass
Evidence mismatch	reject before graph update	false	true	pass

Table 6. Scale sanity run over deterministic controlled streams.

Seed records	Replay records	Active	Logs	Conflicts	Replay	ms/rec.
30	70	40	70	7	0.871	0.024
60	140	80	140	14	0.864	0.039
120	280	160	280	31	0.896	0.030
240	560	320	560	63	0.920	0.028

Table 7. Replay trace excerpt showing representative graph transitions.

Step	Operator	Event transition	Replay note
1	`ADD_EVENT`	E000016	accepted evidence-backed record
2	`ADD_EVENT`	E000007_TS007	accepted evidence-backed record
3	`ADD_EVENT`	E000007	accepted evidence-backed record
19	`MARK_CONFLICT`	E000005_CON005 -¿ E000005	marked unresolved conflict
29	`MERGE_EVENT`	E000007_DUP007 -¿ E000007	merged duplicate into target
32	`UPDATE_SLOT`	E000003_UPD003 -¿ E000003_TS003	updated changed slots

checks verify that failed target resolution and unsupported metadata are logged without being counted as successful graph updates.

4.5. Scale sensitivity and case study

The scale sanity run uses deterministic streams up to 560 replay records. Table 6 reports graph sizes and runtime per record, while Figure 3 combines metadata-hidden agreement with runtime scaling. Table 7 provides a short replay-trace excerpt.

5. Discussion

The oracle-controlled setting is an upper-bound sanity check because controlled metadata can identify intended operators and targets. It should not be read as deployment performance. The metadata-hidden result is more informative for method limits: it shows that simple rules recover part of the injected structure, but still miss many controlled operator labels.

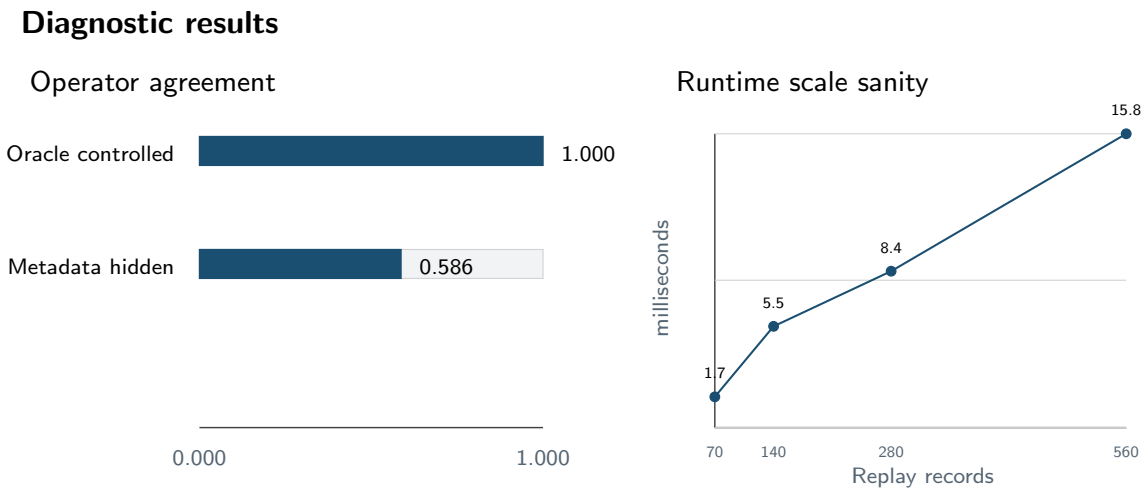

Fig. 3. Diagnostic view of oracle-controlled versus metadata-hidden operator agreement and scale sanity runtime.

The public mini-case is deliberately small and is used only to exercise the replay path on externally sourced text. The exact span gate is strict: it favors auditability and reproducibility over tolerant matching. The artifact reports no extraction F1 and no market prediction. It also does not implement human review, persistent graph storage, or large public-dataset replay. Future work should expand public replay coverage, add human review workflows for unresolved conflicts, and move the graph store to a durable backend.

6. Conclusion

This paper presented an evidence-gated controlled replay prototype for auditable versioned financial event graph updates. The artifact records schema checks, exact evidence containment, target resolution, graph transitions, invariant failures, and version logs. The evidence supports a narrow auditability claim: controlled financial event updates can be replayed into inspectable graph artifacts with explicit success and failure accounting.

References

[1] H. Yang, Y. Chen, K. Liu, Y. Xiao, J. Zhao, DCFEE: A document-level chinese financial event extraction system based on automatically labeled training data, in: Proceedings of ACL 2018, System Demonstrations, 2018, pp. 50–55.
URL https://aclanthology.org/P18-4009/

[2] S. Zheng, W. Cao, W. Xu, J. Bian, Doc2EDAG: An end-to-end document-level framework for chinese financial event extraction, in: Proceedings of EMNLP-IJCNLP 2019, 2019, pp. 337–346.
URL https://aclanthology.org/D19-1032/

[3] R. Ma, T. Gui, L. Li, Q. Zhang, X. Zhou, FewFC: A few-shot financial event extraction dataset, in: Findings of the Association for Computational Linguistics: ACL 2022, 2022, pp. 3566–3577.
URL https://aclanthology.org/2022.findings-acl.282/

[4] T. Lebo, S. Sahoo, D. McGuinness, K. Belhajjame, J. Cheney, D. Corsar, D. Garijo, S. Soiland-Reyes, S. Zednik, J. Zhao, PROV-O: The PROV ontology, W3C Recommendation, accessed: 2026-05-08 (2013).
URL https://www.w3.org/TR/prov-o/

[5] M. Fowler, Event sourcing, Technical article, accessed: 2026-05-08 (2005).
URL https://martinfowler.com/eaaDev/EventSourcing.html

[6] J. Banks, J. S. Carson, B. L. Nelson, D. M. Nicol, Discrete-Event System Simulation, 5th Edition, Pearson, 2010.

[7] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, 2015.

[8] R. G. Sargent, Verification and validation of simulation models, Journal of Simulation 7 (1) (2013) 12–24.

[9] P. Buneman, S. Khanna, W.-C. Tan, Why and where: A characterization of data provenance, in: Proceedings of the 8th International Conference on Database Theory, 2001, pp. 316–330.